

AWS Deployment Guide

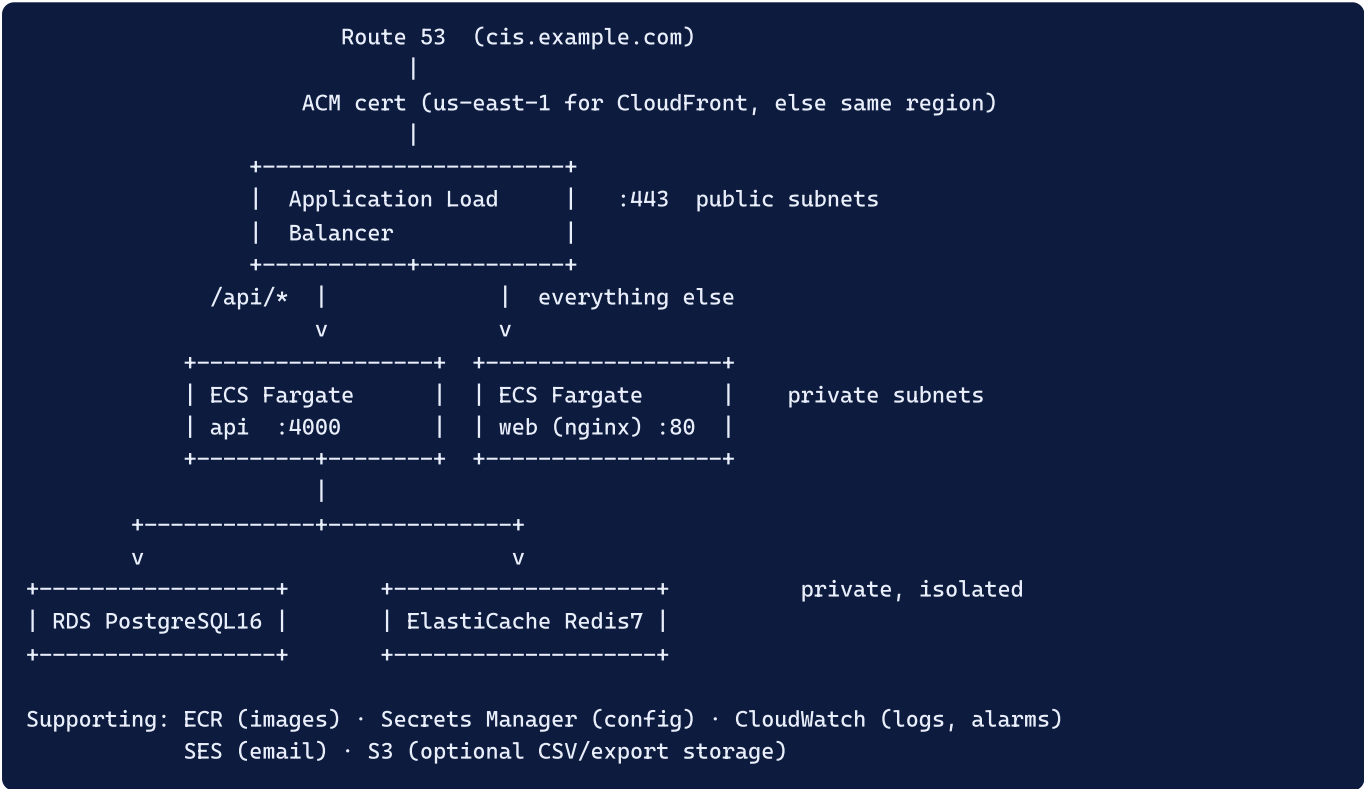
CIS Technologies — work management platform

ECS Fargate · RDS PostgreSQL · ElastiCache Redis · ALB · ECR · Secrets Manager · SES

Read the Deployment Readiness Checklist first.

Two blockers there must be cleared before any of this works: the Prisma migration baseline, and how the browser reaches the API. This guide assumes both are done and uses **one domain with path routing**, which is Option A in that document.

1 · Target architecture



Why this shape

- **One ALB, path-routed.** `/api/*` to the API target group, everything else to the web target group. Single origin, so no CORS and no build-time API URL.
- **Fargate, not EC2.** No servers to patch. Both images already run as non-root with health checks, so they drop straight in.
- **Databases in isolated subnets** with no route to the internet, reachable only from the task security group.

Alternatives, honestly

Option	When it is the better choice
--------	------------------------------

App Runner	Fastest path. Fine for the API, but you still need a VPC connector to reach RDS, and path-routing two services onto one domain is awkward. Good for a demo, weak for production.
Elastic Beanstalk	Only if your team already runs it. Otherwise it hides the parts you will eventually need to tune.
S3 + CloudFront for the web	Cheaper and faster than running an nginx task, but you then need Option C from the checklist (separate API domain, CORS, build-arg). Worth it at scale; more moving parts on day one.
EKS	Only if you already run Kubernetes. Enormous overkill for two containers.

2 · Prerequisites

- ☐ AWS account with billing alerts already configured.
- ☐ AWS CLI v2 installed and `aws sts get-caller-identity` returns your account.
- ☐ A registered domain, ideally in Route 53.
- ☐ An IAM user or role for deployment — **not** the account root.
- ☐ Docker able to build both images locally.
- ☐ A region chosen and used consistently. `ap-south-1` (Mumbai) if your users are in India.

```
export AWS_REGION=ap-south-1
export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
export APP=cis
```

3 · Networking

Three subnet tiers across two Availability Zones. Two AZs is the minimum for an ALB and for RDS Multi-AZ.

Tier	Contains	Internet
Public ×2	ALB, NAT gateway	Internet gateway
Private ×2	ECS tasks (api, web)	Outbound via NAT only
Isolated ×2	RDS, ElastiCache	None

Security groups — the whole matrix

Group	Inbound	From
<code>sg-alb</code>	443, 80	0.0.0.0/0
<code>sg-ecs-web</code>	80	<code>sg-alb</code>
<code>sg-ecs-api</code>	4000	<code>sg-alb</code>
<code>sg-rds</code>	5432	<code>sg-ecs-api</code> only
<code>sg-redis</code>	6379	<code>sg-ecs-api</code> only

Reference security groups, never CIDR ranges, for the database rules.

A CIDR that happens to cover your VPC will quietly expose Postgres to every resource in it.

Cost note: a NAT gateway is roughly \$32/month per AZ plus data charges – often the largest line item in a small deployment. One NAT in a single AZ is an acceptable saving for staging, not for production.

4 · RDS PostgreSQL

```
aws rds create-db-instance \  
  --db-instance-identifier $APP-prod \  
  --engine postgres --engine-version 16 \  
  --db-instance-class db.t4g.micro \  
  --allocated-storage 20 --storage-type gp3 --storage-encrypted \  
  --master-username cisadmin \  
  --manage-master-user-password \  
  --db-subnet-group-name $APP-isolated \  
  --vpc-security-group-ids sg-rds \  
  --backup-retention-period 7 \  
  --multi-az \  
  --no-publicly-accessible \  
  --deletion-protection
```

- `--manage-master-user-password` puts the password straight into Secrets Manager and rotates it — you never handle it.
- `--storage-encrypted` cannot be added later without a snapshot restore. Set it now.
- `--deletion-protection` has saved more production databases than any runbook.
- `db.t4g.micro` suits a pilot. Move to `db.t4g.small` or larger once the dashboard aggregate is under real load.

Create the application user

Do not run the app as the master user. Connect once via a bastion or an ECS exec session:

```
CREATE DATABASE cis;  
CREATE USER cis_app WITH PASSWORD '<from Secrets Manager>';  
GRANT ALL PRIVILEGES ON DATABASE cis TO cis_app;  
\c cis  
GRANT ALL ON SCHEMA public TO cis_app;
```

Connection string, stored as a secret:

```
postgresql://cis_app:PASSWORD@cis-prod.xxxx.ap-south-1.rds.amazonaws.com:5432/cis?sslmode=require
```

5 · ElastiCache Redis

```
aws elasticache create-replication-group \  
  --replication-group-id $APP-redis \  
  --replication-group-description "CIS queue and cache" \  
  --engine redis --engine-version 7.1 \  
  --cache-node-type cache.t4g.micro \  
  --num-cache-clusters 2 \  
  --automatic-failover-enabled \  
  --transit-encryption-enabled \  
  --cache-subnet-group-name $APP-isolated \  
  --security-group-ids sg-redis
```

With transit encryption on, the URL scheme is `rediss://` — two s.

`redis://` will fail to connect, and the error is not obvious.

6 · ECR and images

```
aws ecr create-repository --repository-name $APP-api
aws ecr create-repository --repository-name $APP-web

aws ecr get-login-password --region $AWS_REGION \
  | docker login --username AWS --password-stdin $ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com

SHA=$(git rev-parse --short HEAD)
REG=$ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com

docker build -f apps/api/Dockerfile -t $REG/$APP-api:$SHA .
docker build -f apps/web/Dockerfile -t $REG/$APP-web:$SHA .
docker push $REG/$APP-api:$SHA
docker push $REG/$APP-web:$SHA
```

Tag with the commit SHA, always.

`latest` makes rollback impossible and makes "which build is running?" unanswerable. Enable ECR scan-on-push and a lifecycle policy that expires untagged images after 14 days.

Building on an Apple Silicon Mac? Add `--platform linux/amd64`, or your tasks will fail to start with an exec format error.

7 · Secrets Manager

```
aws secretsmanager create-secret --name $APP/prod/api --secret-string '{
  "DATABASE_URL": "postgresql://cis_app:...@...:5432/cis?sslmode=require",
  "REDIS_URL": "rediss://cis-redis...:6379",
  "JWT_ACCESS_SECRET": "...48 random bytes...",
  "JWT_REFRESH_SECRET": "...different 48 random bytes...",
  "VAPID_PUBLIC_KEY": "...",
  "VAPID_PRIVATE_KEY": "...",
  "SMTP_USER": "...",
  "SMTP_PASSWORD": "..."
}'
```

In the task definition, reference each key individually under `secrets` so values never appear in the definition body, and keep non-sensitive values in `environment`:

```
"secrets": [
  { "name": "DATABASE_URL",
    "valueFrom": "arn:aws:secretsmanager:...:secret:cis/prod/api:DATABASE_URL:~" }
],
"environment": [
  { "name": "NODE_ENV",      "value": "production" },
  { "name": "WEB_BASE_URL",  "value": "https://cis.example.com" },
  { "name": "API_BASE_URL",  "value": "https://cis.example.com" },
  { "name": "EMAIL_DRIVER",  "value": "smtp" },
  { "name": "SMTP_HOST",     "value": "email-smtp.ap-south-1.amazonaws.com" },
  { "name": "SMTP_PORT",     "value": "587" },
  { "name": "SMTP_SECURE",   "value": "false" }
]
```

The task **execution** role needs `secretsmanager:GetSecretValue` on that ARN, plus the managed `AmazonECSTaskExecutionRolePolicy`.

8 · ALB, certificate and DNS

1. Request an ACM certificate for `cis.example.com` in your application region and validate by DNS.
2. Create an internet-facing ALB in the public subnets with `sg-alb`.
3. Create two target groups, both **type IP** (required for Fargate awsipc networking):

Target group	Port	Health check
<code>tg-api</code>	4000	<code>/api/v1/health/ready</code>
<code>tg-web</code>	80	<code>/</code>

4. HTTPS :443 listener — default action forward to `tg-web`; add rule **path is** `/api/*` → **forward to** `tg-api` at priority 1.
5. HTTP :80 listener — redirect permanently to HTTPS.
6. Route 53 A-record alias for `cis.example.com` pointing at the ALB.

Use `/health/ready` as the health check, not `/health/live`.

Ready verifies the database and Redis are actually reachable; live only proves the Node process exists, so a task with a broken database would keep receiving traffic.

9 · ECS cluster, tasks and services

```
aws ecs create-cluster --cluster-name $APP-prod \  
  --capacity-providers FARGATE --settings name=containerInsights,value=enabled
```

Task	Size	Notes
<code>cis-api</code>	0.5 vCPU / 1 GB	Port 4000. Desired count 2 for zero-downtime deploys.
<code>cis-web</code>	0.25 vCPU / 0.5 GB	Port 80. Static nginx; 2 tasks is ample.
<code>cis-migrate</code>	0.5 vCPU / 1 GB	Same image as the API, command overridden. Run once per release, never as a service.

Log configuration on every task:

```
"logConfiguration": {  
  "logDriver": "awslogs",  
  "options": {  
    "awslogs-group": "/ecs/cis-api",  
    "awslogs-region": "ap-south-1",  
    "awslogs-stream-prefix": "ecs",  
    "awslogs-create-group": "true"  
  }  
}
```

Create both services in the private subnets, attached to their target groups, with `--health-check-grace-period-seconds 60` so the API is not killed while it boots.

Running the migration

```
aws ecs run-task --cluster $APP-prod \  
  --task-definition cis-migrate --launch-type FARGATE \  
  --network-configuration "awsvpcConfiguration={subnets=[subnet-a,subnet-b],securityGroups=[sg-ecs-  
api]}" \  
  --overrides '{"containerOverrides":[{"name":"migrate",  
    "command":["npm","prisma","migrate","deploy"]}]}'
```

Wait for the task to reach `STOPPED` with exit code 0 before deploying the API. If it exits non-zero, stop — do not roll the API forward onto a half-migrated schema.

10 • Email with SES

1. Verify your sending domain in SES and publish the DKIM records it gives you.
2. Add SPF and a DMARC policy. Without them your invitations land in spam.
3. **Request production access** — the sandbox only sends to verified addresses, which will look like "invitations are broken" during your first test.
4. Create SMTP credentials (these are *not* your IAM keys) and store them in the secret.
5. Set `EMAIL_DRIVER=smtp`, host `email-smtp.<region>.amazonaws.com`, port 587.

11 • First run

1. Confirm the migration task succeeded.
2. `curl https://cis.example.com/api/v1/health/ready` returns `"database":true,"cache":true`.
3. Create the first owner. **Do not run the seed.** Either register through `POST /api/v1/auth/register` once (it creates the organisation and an OWNER), or insert one membership manually.
4. Sign in, create the real workspace and boards.
5. Invite the rest of the team, or let them use Request access and approve them.
6. Work through the smoke tests in §7 of the checklist.

12 · CI/CD with GitHub Actions

Use OIDC — no long-lived AWS keys in GitHub.

```
permissions: { id-token: write, contents: read }

steps:
  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: arn:aws:iam::ACCOUNT:role/github-deploy
      aws-region: ap-south-1

  - uses: aws-actions/amazon-ecr-login@v2

  - name: Build and push
    run: |
      SHA=${GITHUB_SHA::7}
      docker build -f apps/api/Dockerfile -t $REG/cis-api:$SHA .
      docker build -f apps/web/Dockerfile -t $REG/cis-web:$SHA .
      docker push $REG/cis-api:$SHA
      docker push $REG/cis-web:$SHA

  - name: Migrate
    run: aws ecs run-task --cluster cis-prod --task-definition cis-migrate ...

  - name: Deploy
    run: |
      aws ecs update-service --cluster cis-prod --service cis-api --force-new-deployment
      aws ecs update-service --cluster cis-prod --service cis-web --force-new-deployment
```

Trust policy on `github-deploy` must pin the repository and branch, or any repo on GitHub can assume it:

```
"Condition": {
  "StringEquals": { "token.actions.githubusercontent.com:aud": "sts.amazonaws.com" },
  "StringLike": { "token.actions.githubusercontent.com:sub":
    "repo:YOUR_ORG/YOUR_REPO:ref:refs/heads/main" }
}
```

13 · Monitoring and alarms

Alarm	Threshold
ALB HTTPCode_Target_5XX_Count	> 10 in 5 min
ALB UnHealthyHostCount	≥ 1 for 2 min
ALB TargetResponseTime p95	> 1s for 5 min
RDS CPUUtilization	> 80% for 10 min
RDS DatabaseConnections	> 80% of max
RDS FreeStorageSpace	< 20%

ElastiCache DatabaseMemoryUsagePercentage	> 75%
ECS RunningTaskCount	< desired for 5 min

Send them all to one SNS topic with a real email or pager attached. An alarm nobody receives is decoration.

Auto scaling

```
aws application-autoscaling register-scalable-target \
  --service-namespace ecs --scalable-dimension ecs:service:DesiredCount \
  --resource-id service/cis-prod/cis-api --min-capacity 2 --max-capacity 6
# then a target-tracking policy at 65% average CPU
```

14 · Backup and recovery

- ☐ RDS automated backups retained at least 7 days (30 for anything regulated).
- ☐ **Restore rehearsed at least once** into a scratch instance. Time it — that number is your actual RTO, whatever the plan claims.
- ☐ Final snapshot before any destructive change.
- ☐ Deletion protection on in production.
- ☐ Redis needs no backup: it holds the queue and cache only, and rebuilds on its own.
- ☐ ECR lifecycle policy keeps recent tagged images so rollback stays possible.

15 · Indicative monthly cost

Item	Small	Production
ECS Fargate (api + web)	~\$25	~\$70
RDS PostgreSQL	~\$15 single-AZ	~\$60 Multi-AZ
ElastiCache Redis	~\$12	~\$30 with replica
ALB	~\$18	~\$22
NAT gateway	~\$32 one AZ	~\$65 two AZ
Data transfer, logs, ECR, SES	~\$10	~\$30
Approximate total	~\$110/mo	~\$280/mo

ap-south-1, on-demand, mid-2026 list prices. Verify against the AWS pricing calculator before committing. Compute Savings Plans cut the Fargate line by roughly 20–30%; VPC endpoints for ECR and Secrets Manager can remove much of the NAT cost.

16 · Common failures and what they mean

Symptom	Cause
Task starts then stops, exit 1, no logs	Env validation failed at boot. Check the log group — the missing variable is named.

Target group permanently unhealthy	Health path wrong, security group blocks the ALB, or grace period too short for boot.
API cannot reach the database	<code>sg-rds</code> not referencing <code>sg-ecs-api</code> , or the task is in a subnet with no route.
Redis connection timeout	<code>redis://</code> used where transit encryption requires <code>rediss://</code> .
App loads, every API call 404s	The <code>/api/*</code> listener rule is missing or below the default rule in priority.
App loads, every API call CORS-blocked	<code>WEB_BASE_URL</code> does not exactly match the browser origin.
Login works, tables are empty	Migration never ran. This is blocker 1 from the checklist.
Invitations never arrive	SES still in sandbox, or <code>EMAIL_DRIVER</code> left as <code>console</code> .
<code>exec format error</code>	Image built on ARM without <code>--platform linux/amd64</code> .

17 • Teardown

```
aws ecs update-service --cluster cis-prod --service cis-api --desired-count 0
aws ecs update-service --cluster cis-prod --service cis-web --desired-count 0
aws ecs delete-service --cluster cis-prod --service cis-api --force
aws ecs delete-service --cluster cis-prod --service cis-web --force
aws rds delete-db-instance --db-instance-identifier cis-prod \
    --final-db-snapshot-identifier cis-final-$(date +%Y%m%d)
aws elasticache delete-replication-group --replication-group-id cis-redis
# then: ALB, NAT gateway, VPC, ECR repositories, secrets
```

Delete the NAT gateway explicitly.

It bills hourly whether or not anything uses it, and it survives deleting the services that needed it.